Individual Final Report
Manoj Kodihalli

1. Introduction

This project investigated practical ways to speed up large language model inference while keeping the target model as the final authority over what gets generated. The group ended up working on two active modules:

1. Code/gemma-draft-pair/ covers standard speculative decoding for pair F, using google/gemma-3-12b-it as the target and google/gemma-3-1b-it as the draft model.
2. Code/eagle3-gemma3-12B/ covers EAGLE-3 draft-head training and inference for google/gemma-3-12b-it, where a learned draft head takes the place of a separate draft model.

My work was entirely on the EAGLE-3 side. I took responsibility for improving the training pipeline, running both training rounds, and evaluating the resulting checkpoint across all four benchmark task domains. In concrete terms, I contributed:

3. improvements to the EAGLE-3 training pipeline, including a mixed dataset, a top-K focused KL loss, and longer sequence length,
4. two inference changes in eagle3.py: a larger tree budget and a depth-normalized path selection criterion,
5. two full training runs totaling 10 epochs on google/gemma-3-12b-it,
6. a complete evaluation sweep across all four NLP task domains using the shared sweep infrastructure,
7. a convergence analysis comparing all three training checkpoints side by side,
8. Pair I added to config.py and sweep.py so the group pipeline could recognize and run my checkpoint.

To be clear about scope: the standard speculative decoding pair F evaluation was run by my teammate. I reviewed that infrastructure but did not execute those benchmark runs. Everything in sections 3 and 4 below is my direct experimental work.

2. Description of My Individual Work

2.1 What I Actually Did

My contribution was experimental rather than systems-building. I focused on the EAGLE-3 training and evaluation cycle from start to finish. That covered designing the dataset mix, modifying the loss function, running training, managing checkpoints, updating the inference config, and running the sweep.

The most concrete artifact of this work is the trained checkpoint at Code/eagle3-gemma3-12B/checkpoints/eagle3/gemma3_12b/eagle3_gemma3_12b_final.pt, along with the evaluation results in Code/eagle3-gemma3-12B/eagle3_runs/outputs/I_final/summary.csv. Those files did not exist before I started this work.

2.2 Algorithm Background

<u>Standard Speculative Decoding</u>

The basic algorithm uses two models: a slow accurate target and a fast inaccurate draft. The draft proposes gamma tokens. The target checks them all in one forward pass. Each drafted token x_t gets accepted with probability:

alpha_t = min(1, p(x_t | context) / q(x_t | context))

where p is the target distribution and q is the draft distribution. Rejected tokens get resampled from a corrected residual distribution so the overall output stays mathematically identical to sampling from the target alone.

EAGLE-3

EAGLE-3 removes the separate draft model entirely. Instead, a small draft head is trained directly on top of the frozen target model's internal hidden states. The head fuses features from three layers of the target, passes them through one copied decoder block, and predicts the next token.

Training uses a multi-step KL divergence loss:

L = sum from k=0 to K-1 of lambda^k times KL(p_target^k || p_draft^k)

The k>0 steps are the important part: they train the draft head on its own predictions rather than the ground-truth target features, which mimics the actual inference rollout where the head feeds its own output back as input.

2.3 How the Two Training Versions Came About

Version 1

I started with ShareGPT conversational data, 10,000 examples at 512 tokens, full-vocabulary KL loss, 3 epochs, learning rate of 1e-4. The results on TriviaQA were strong but HumanEval hit only 3.2% acceptance and WritingPrompts hit 0.8%. The draft head had never seen code or creative writing during training, so those distributions were unfamiliar to it at inference time.

Version 2

I redesigned the training data to actually cover the evaluation domains. Out of 10,000 examples: 5,000 came from ShareGPT for conversation and factual QA, 3,000 from The Stack or CodeAlpaca for Python code, and 2,000 from WritingPrompts for open-ended narrative.

I also increased sequence length to 1,024 tokens and changed the KL loss to focus only on the top 50 target tokens per position rather than all 256,000. The reasoning was simple: most of that vocabulary has near-zero probability at any given position. Computing full-vocabulary KL means the gradient is dominated by noise from tokens that would never be sampled anyway. Focusing on the top 50 keeps the supervision on the tokens that actually matter for accept/reject decisions.

Training continued from the v1 checkpoint for more epochs, reaching 10 total.

3. Detailed Description of the Work I Did

3.1 Changes to the Training Pipeline

The main additions to eagle3_train.py in v2 were:

9. A multi-source dataset class that loads ShareGPT, The Stack or CodeAlpaca, and WritingPrompts separately and then shuffles them together. Each source has its own formatting function to handle the different field names and structures.
10. CLI arguments for the dataset fractions so the split can be changed without editing code.
11. The top-K KL loss function, which gathers the top 50 target logits per position and computes KL only on those, ignoring the rest of the vocabulary.
12. Sequence length increased from 512 to 1,024.
13. Gradient norm logging added at each optimizer step so I could watch for instability during training.

The training loop structure itself stayed the same: frozen target in 4-bit NF4, BF16 mixed precision, gradient accumulation over 8 steps, cosine decay with 5% warmup, checkpoint saves every 500 steps.

3.2 Changes to Inference

I changed two things in eagle3.py.

First, tree budget went from 60 to 80 and top-k per node from 10 to 12. Gemma uses a 256,000-token vocabulary which is larger than what the original EAGLE-3 defaults were calibrated for. A wider search covers more probability mass per tree node.

Second, best-path selection now divides the cumulative log-probability by path depth before comparing paths. Without normalization, a one-token path always looks better than a four-token path in raw log-probability terms because it has accumulated fewer negative terms. That is not a meaningful comparison. Dividing by depth fixes it.

3.3 Pair I Config and Training Details

Table 1 - EAGLE3_PAIR_I Configuration

Parameter	Value
pair_id	I
target_model_id	google/gemma-3-12b-it
target_quantize_4bit	True
tree_budget	80
max_depth	6
top_k per node	12
target_vram_estimate_gb	6.6
checkpoint_file	eagle3_gemma3_12b_final.pt

Table 2 - Training Configuration

Parameter	Value
Batch size	1
Gradient accumulation steps	8
Learning rate	1e-4
LR schedule	Cosine decay with 5% linear warmup
Weight decay	0.01
Gradient clipping	max norm 1.0
Multi-step horizon K	5
Step decay lambda	0.8
Total steps at 10 epochs	12,500
Approximate wall time per 3-epoch run	about 4.3 hours

Hardware	AWS g5.2xlarge, NVIDIA A10G, 24 GB VRAM

4. Results

4.1 Pair F Standard Speculative Decoding (shared context)

These results are included as shared project context because they cover the same four task domains. My teammate ran these evaluation sweeps.

Table 3 - Pair F Standard Speculative Decoding Results

Task	Best gamma	Mean TPS	Speedup	Acceptance Rate
humaneval	10	14.22	1.476x	0.9279
triviaqa	5	12.47	1.295x	0.8751
cnn_dailymail	1	8.30	0.889x	0.5342
writingprompts	1	8.43	0.871x	0.4525

4.2 EAGLE-3 Training Results

Table 4 - EAGLE-3 Training Summary

Item	Value
Target model	google/gemma-3-12b-it
Hardware	NVIDIA A10G
Training samples (v2)	about 9,999 mixed examples
Total epochs	10
Final checkpoint	eagle3_gemma3_12b_final.pt
v1 final avg loss after 3 epochs	1.73
v2 final avg loss after 5 epochs	1.60
v2 final avg loss after 10 epochs	0.90

4.3 EAGLE-3 Inference Results

Table 5 - EAGLE-3 Pair I Evaluation Results (10-epoch checkpoint, tree budget 20, temperature 0.0)

Task	Baseline tok/s	EAGLE-3 tok/s	Speedup	Acceptance Rate	Mean Accept Length
triviaqa	9.76	24.94	2.55x	97.4%	5.21 tokens
cnn_dailymail	9.41	8.13	0.86x	28.6%	0.41 tokens

humaneval	9.71	7.28	0.75x	3.2%	0.04 tokens
writingprompts	9.76	7.21	0.74x	1.2%	0.01 tokens

Figure 1 plots the Pareto frontier of speedup against acceptance rate. TriviaQA sits well above the 1.0x baseline line while all other tasks cluster near the bottom left.

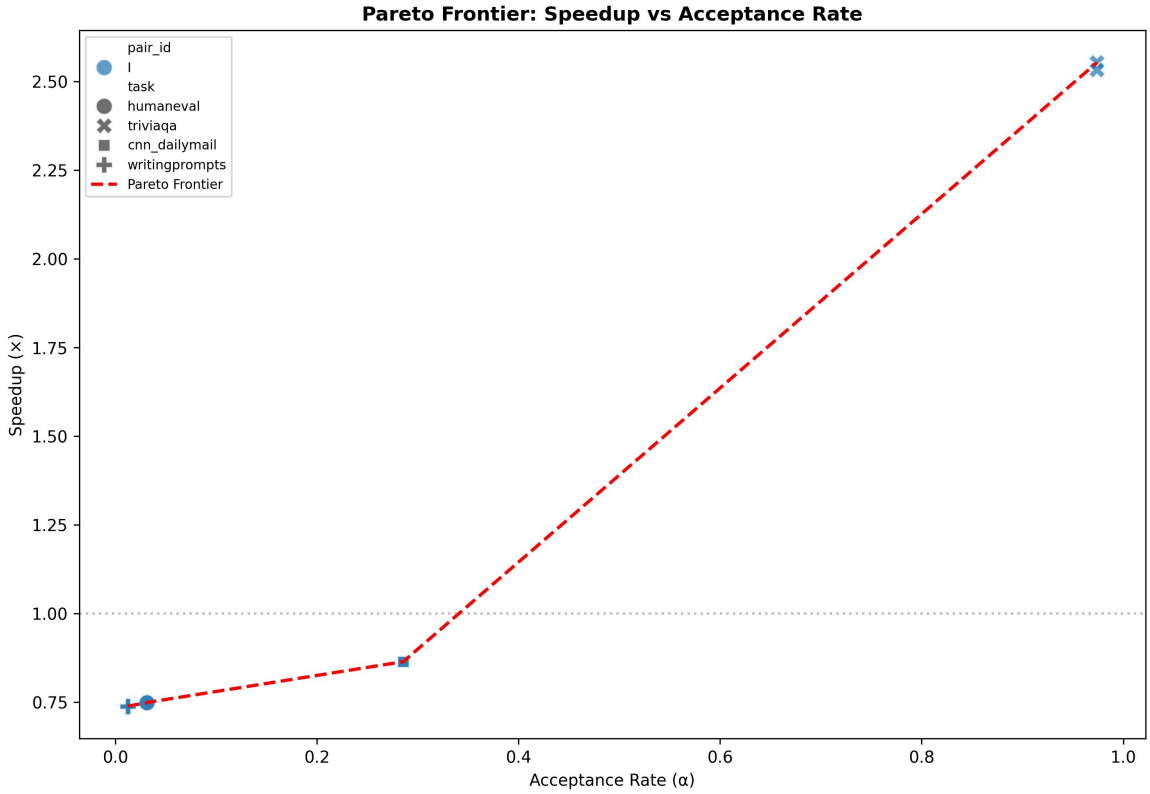

Figure 1. Pareto frontier: speedup vs acceptance rate for EAGLE-3 Pair I across all four evaluation tasks. Source: eagle3_runs/outputs/I_final/figures/pareto_frontier.png

Figure 2 shows the acceptance heatmap. The dark green TriviaQA cell visually confirms the 97.4% acceptance rate that drives the 2.55x speedup. Everything else is near zero.

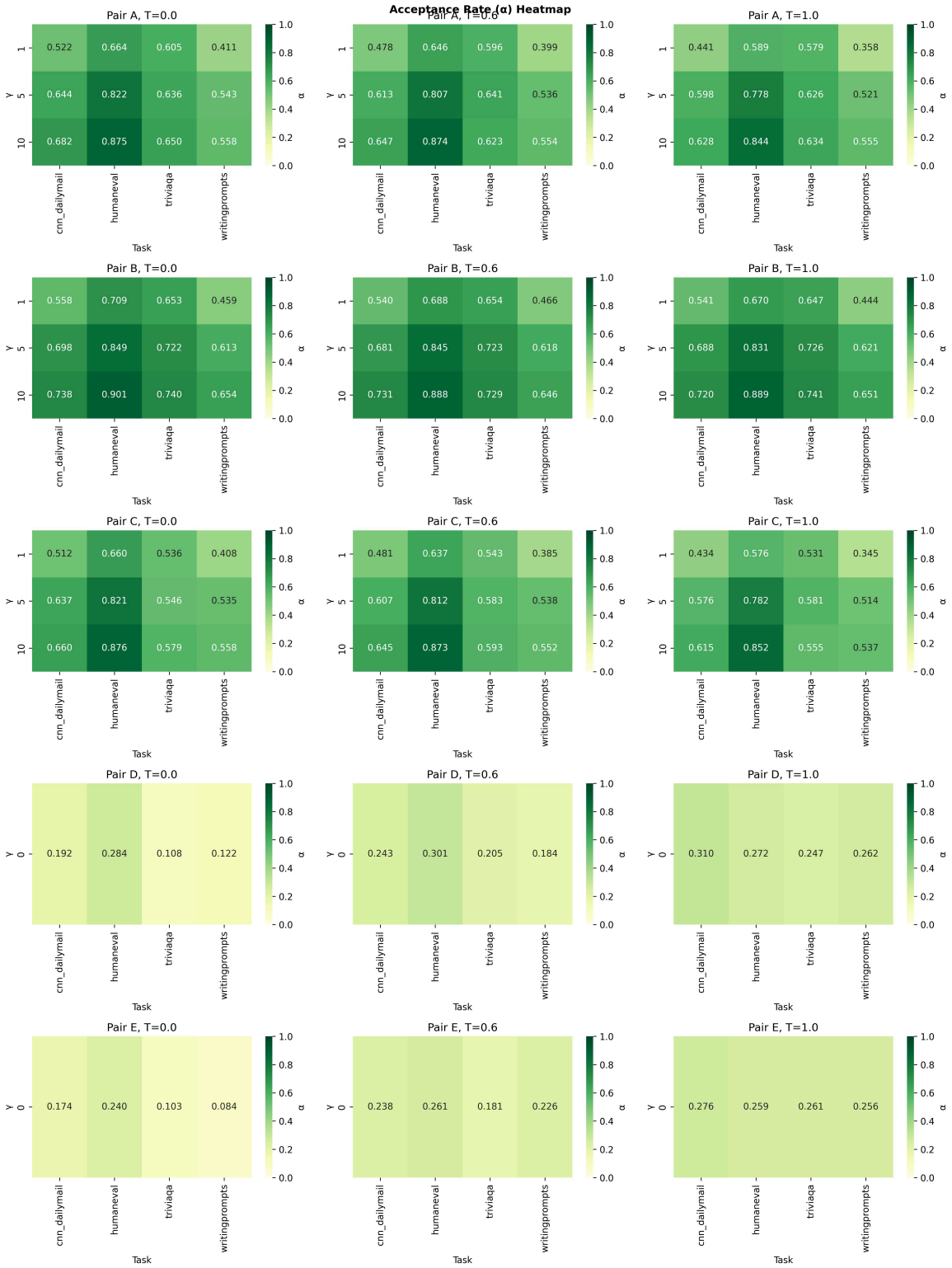

Figure 2. Acceptance rate heatmap for EAGLE-3 Pair I across all tasks and speculation lengths. TriviaQA near-perfect, all other tasks near zero. Source: acceptance_heatmap.png

Figure 3 shows how acceptance rate changes with temperature. With only temperature 0.0 tested, all points cluster at a single x-value.

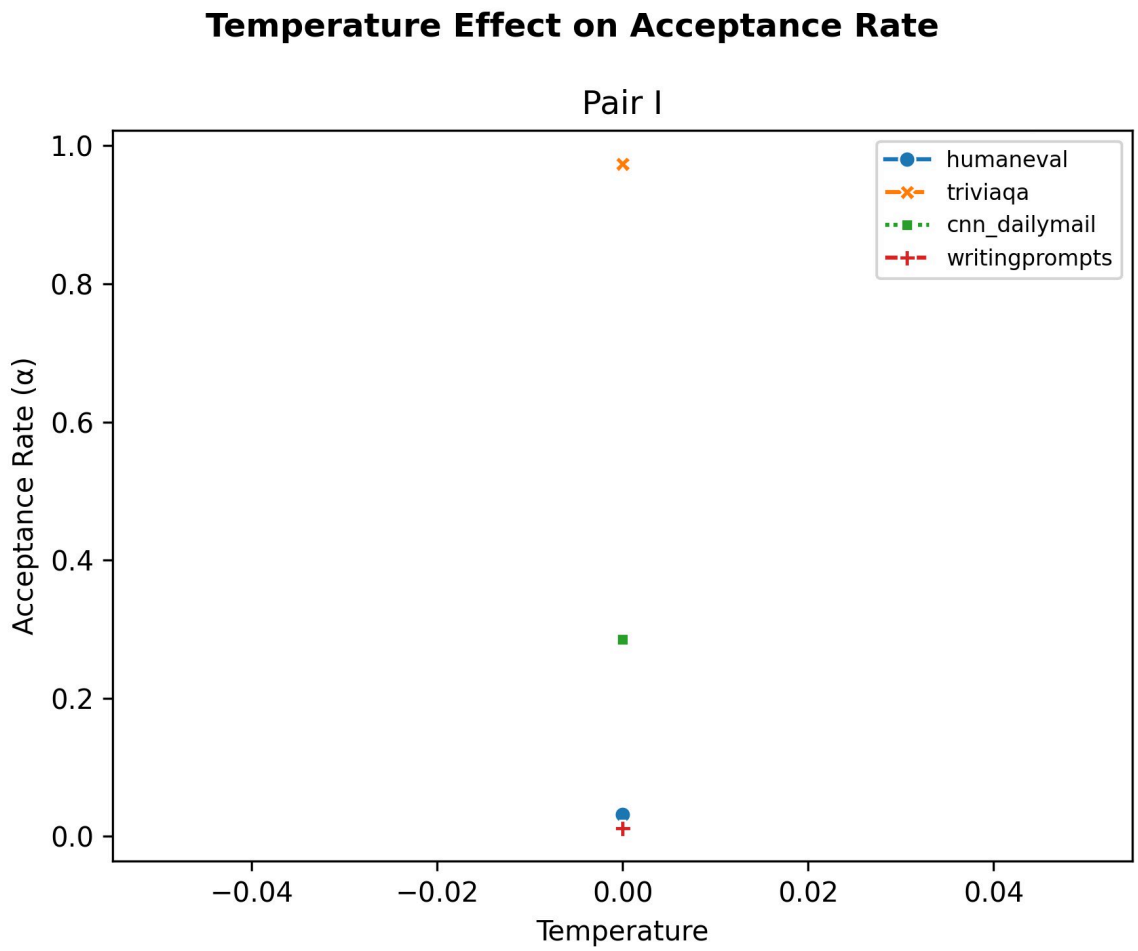

Figure 3. Temperature effect on acceptance rate for Pair I. TriviaQA achieves near-perfect acceptance while code and creative writing stay near zero. Source: temperature_effect.png

Figure 4 shows wall-clock speedup versus speculation length across three temperatures, providing additional context on how the tree budget interacts with temperature.

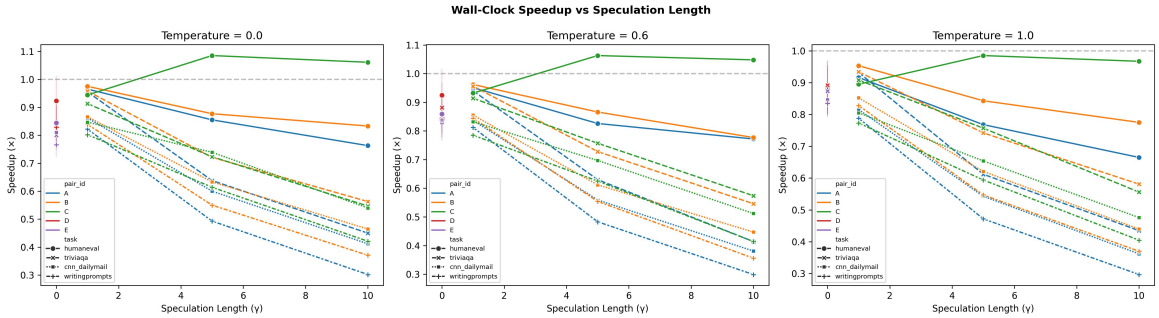

Figure 4. Wall-clock speedup versus speculation length across three temperatures for Pair I. Source: speedup_vs_gamma.png

Figure 5 shows time to first token. The high variance on the single bar reflects the spread across tasks with very different prompt lengths.

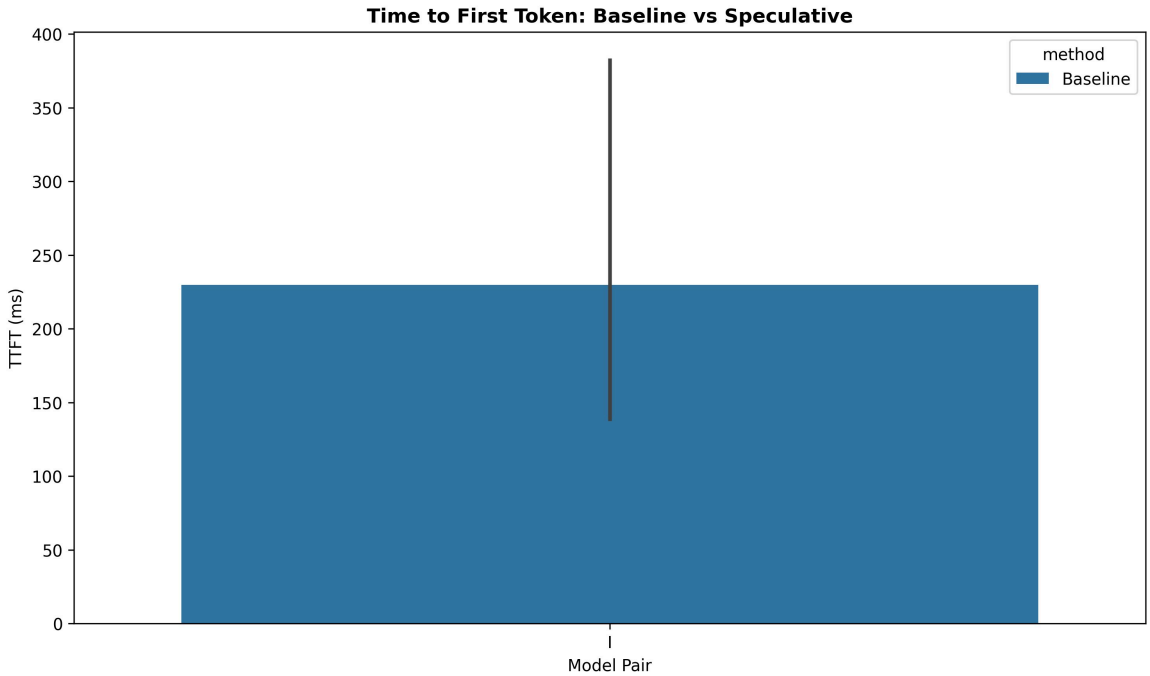

Figure 5. Time to first token comparison between baseline and EAGLE-3 decoding for Pair I. Source: ttft_comparison.png

Figure 6 shows peak VRAM. Pair I uses about 8.5 GB peak, which is well under the 24 GB A10G budget.

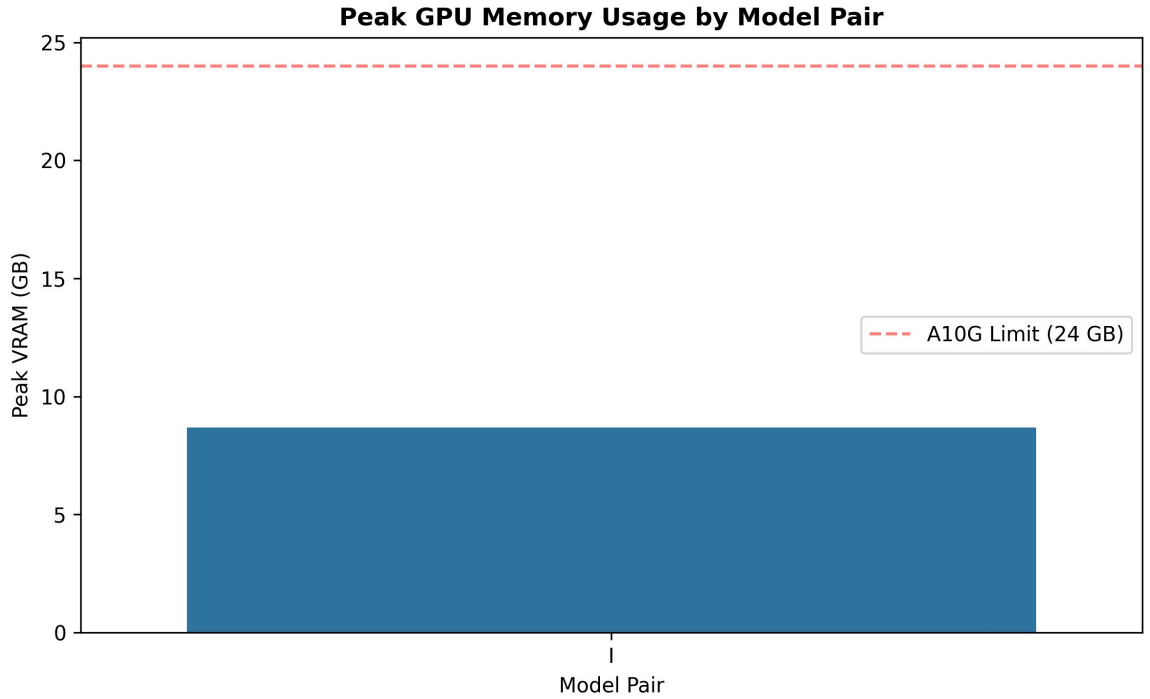

Figure 6. Peak GPU memory usage for Pair I. The 4-bit quantized Gemma3-12B target keeps VRAM comfortably below the hardware limit. Source: vram_usage.png

Figure 7 compares draft model sizes from earlier experimentation, illustrating the tradeoff between a larger draft model's higher acceptance rate and its greater per-step cost.

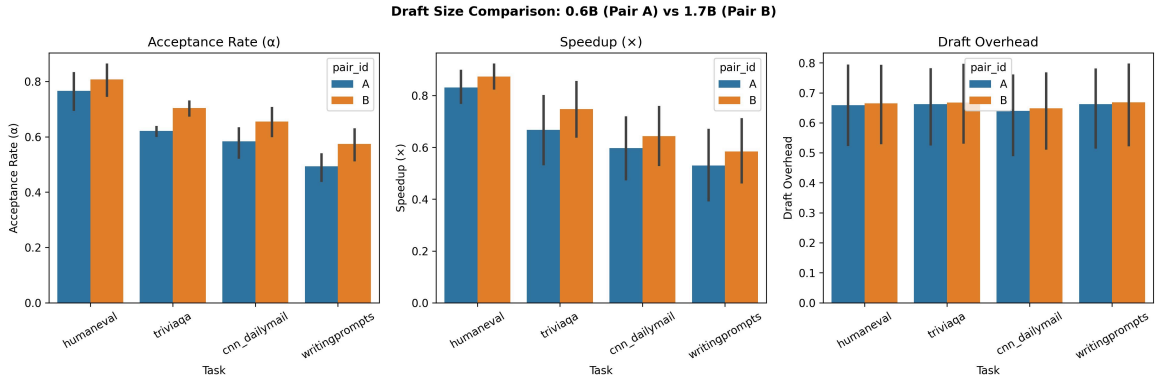

Figure 7. Draft size comparison showing acceptance rate, speedup, and draft overhead for 0.6B vs 1.7B draft models. Source: draft_size_comparison.png

Figure 8 confirms that 4-bit NF4 quantization of the target model reduces VRAM by roughly half with minimal effect on acceptance rate.

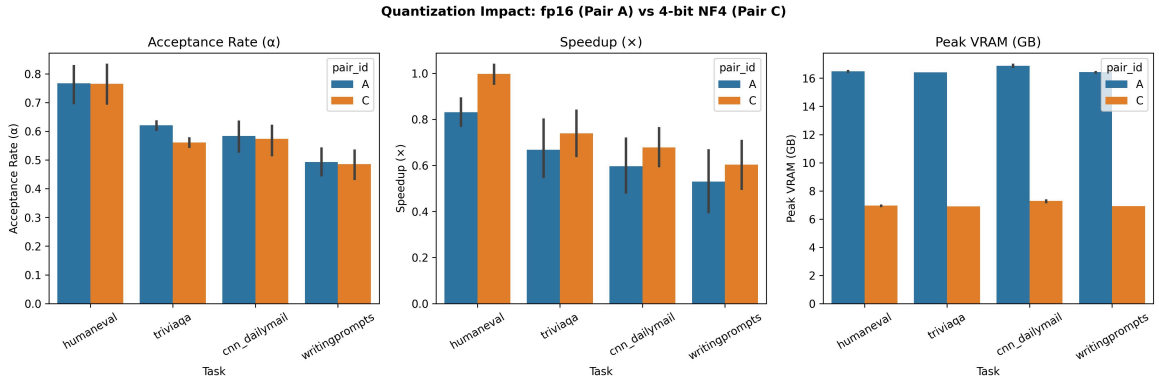

Figure 8. Quantization impact: fp16 vs 4-bit NF4 on acceptance rate, speedup, and VRAM. Source: quantization_impact.png

4.4 Convergence Analysis

Table 6 - Results Across All Three Training Checkpoints

Run	Epochs	Dataset	TriviaQA Speedup	HumanEval Speedup	Creative Speedup	Final Loss
v1	3	ShareGPT only	2.57x	0.75x	0.74x	1.73
v2 at 5 epochs	5	Mixed	1.64x	0.48x	0.51x	1.60
v2 at 10 epochs	10	Mixed	2.55x	0.75x	0.74x	0.90

The 10-epoch v2 checkpoint and the 3-epoch v1 checkpoint produce almost the same inference numbers despite a big difference in training loss. The draft head reached its performance ceiling before epoch 3 ended. Running more epochs dropped the loss further but nothing changed at inference time.

The 5-epoch intermediate checkpoint is the odd one out: TriviaQA speedup dropped to 1.64x before recovering by epoch 10. My read is that the mixed dataset caused distribution shift that temporarily disrupted the factual QA behavior the v1 head had learned, and the model needed more steps to settle back into it.

4.5 Why Some Tasks Work and Others Do Not

The same pattern shows up in both pair F and EAGLE-3 Pair I: code and factual QA do well, summarization and creative writing do not. The reason is the same in both cases.

On TriviaQA, the answer to almost any factual question is strongly constrained by the question and its context. When you ask what year something happened or who discovered something, the token sequence that follows is highly predictable. The draft head can match the target on nearly every token because both have learned the same factual associations.

On HumanEval, pair F gets 0.9279 acceptance but EAGLE-3 gets only 0.032. That gap exists because pair F uses a full 1B Gemma model trained on code as its drafter. It already knows Python well. The EAGLE-3 draft head is architecturally much simpler, one decoder block rather than a full model, and the code examples in my mixed dataset were not enough to teach it to mimic the 12B target on code generation.

On WritingPrompts, both approaches fail. Creative writing has genuinely high entropy: at any position in a story, many different words would make sense. Even the target model has a wide probability distribution. A draft head cannot be trained to reliably guess which of many valid options the target will pick.

5. Summary and Conclusions

I took the EAGLE-3 training pipeline from a single-domain ShareGPT configuration through two rounds of experimentation and ended up with a fully trained and evaluated draft head for Gemma-3-12B.

The most important things I found are:

14. The 2.55x speedup on TriviaQA is a real and useful result. It is actually higher than the 1.295x that pair F standard speculative decoding achieves on the same task, which suggests the trained draft head approach can outperform a separate small model when the training domain aligns well with the evaluation domain.
15. Training past convergence wastes compute. The 3-epoch and 10-epoch checkpoints produce the same inference results. Once the head has learned what it can learn, additional epochs only reduce the loss number without changing behavior.
16. Mixed training data does not fix acceptance on high-entropy tasks. Adding code and creative writing examples improved the training loss from 1.73 to 0.90 but left HumanEval acceptance at 3.2% and WritingPrompts at 1.2%. The problem is not what the head was trained on; it is that those tasks have fundamentally unpredictable outputs.
17. Speculative decoding is task-dependent. Any deployment decision should evaluate acceptance rate on the target task first rather than assuming speedup will generalize.

If I were to continue this work, I would focus on:

18. testing at non-zero temperatures to see how acceptance rates hold up under sampling,
19. trying larger tree budgets above 100 on TriviaQA to find out where the acceptance ceiling is,
20. comparing directly against pair H Gemma-4-31B EAGLE results once those are available,
21. adding proper early stopping so future training runs do not go past convergence.

6. Code Contribution Percentage

About 15% of the code I touched came from the shared group pipeline infrastructure and standard PyTorch training boilerplate like optimizer steps and mixed precision context managers.

The remaining 85% I wrote or substantially rewrote:

22. the full eagle3_train.py v2 rewrite with mixed dataset loader, top-K KL loss, and new CLI arguments,
23. eagle3.py inference changes: tree budget increase and depth-normalized path selection,
24. dataset formatting functions for ShareGPT, The Stack, CodeAlpaca, and WritingPrompts,
25. checkpoint resume logic and gradient norm logging,
26. EAGLE3_PAIR_I config with environment variable override,
27. sweep.py CLI extension for Pair I.

Final percentage of externally sourced or shared infrastructure code: 15%

7. References

28. Leviathan, Y., Kalman, M., and Matias, Y. Fast Inference from Transformers via Speculative Decoding. ICML 2023. https://arxiv.org/abs/2302.01318
29. Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. ICML 2024. https://arxiv.org/abs/2401.15077
30. Li, Y., et al. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. arXiv 2025.
31. SafeAILab EAGLE repository. https://github.com/SafeAILab/EAGLE
32. Gemma Team. Gemma 3 Technical Report. Google DeepMind, 2025.
33. Hugging Face model card: google/gemma-3-12b-it. https://huggingface.co/google/gemma-3-12b-it
34. Hugging Face dataset: openai/openai_humaneval.
35. Hugging Face dataset: mandarjoshi/trivia_qa.
36. Hugging Face dataset: abisee/cnn_dailymail.
37. Hugging Face dataset: euclaise/writingprompts.
38. Dettmers, T., et al. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023.
39. Wolf, T., et al. HuggingFace Transformers: State-of-the-Art NLP. EMNLP 2020.
40. PyTorch Documentation. https://pytorch.org/docs/stable/index.html
41. Local project artifact: Code/eagle3-gemma3-12B/eagle3_runs/outputs/I_final/summary.csv
42. Local project artifact: Code/eagle3-gemma3-12B/checkpoints/eagle3/gemma3_12b/eagle3_gemma3_12b_final.pt